Phineas: Robot Móvil Recolector de Cubos para un Sistema Multirrobot Colaborativo sobre ROS 2

Documento técnico de diseño

[Nombre Apellido], [Nombre Apellido], [Nombre Apellido] y [Nombre Apellido]

Equipo Phineas

Escuela de Ingeniería Informática, Universidad Católica Andrés Bello

Robótica

Prof. [Nombre del profesor]

12 de octubre de 2026

Índice

Introducción 4
Planteamiento del Problema 4
Contexto 4
Problema Específico de Phineas 5
Objetivos 5
Restricciones y Supuestos 6
Descripción General del Sistema 7
Stack Tecnológico Propuesto 9
Arquitectura de Hardware 11
Componentes Disponibles y Mejoras Futuras 11
Diagrama de Bloques 12
Restricciones Eléctricas Verificadas 13
Mecanismo de Recolección 14
Asignación de Pines y Conexiones 17
Control de los Motores 18
Conexión del Sensor de Línea y de los Motores 19
Arquitectura de Software 20
Nodos de ROS 2 20
Percepción de Cubos por Color 22
Cinemática Mecanum 22
Máquina de Estados 23
Maniobra de Entrega 24
Convivencia en el Espacio Compartido 25
Solo Avanzar 27
Frenado y Giro ante el Otro Robot 28
Comunicación con Ferb y Perry 29
Red y Descubrimiento 29
Contrato de Interfaces de Phineas 30
Protocolo de Acceso a la Zona de Transferencia 30
Simulación en Webots 32
Del Simulador al Robot Real 33
Riesgos y Verificaciones Pendientes 35
Plan de Pruebas 35

Conclusiones 36
Historial de Cambios 37
Referencias 38

Phineas: Robot Móvil Recolector de Cubos para un Sistema Multirrobot Colaborativo sobre ROS 2

Introducción

El proyecto de Robótica de este período consiste en un sistema de tres robots que cooperan sin una computadora central. Dos robots móviles, Phineas y Ferb, recogen cubos de colores en una pista que comparten, y un brazo robótico fijo, Perry, los toma de una zona de transferencia y los clasifica. Cada robot tiene su propia Raspberry Pi 5 y todos se comunican sobre ROS 2 (Robot Operating System 2, el middleware de robótica que gestiona procesos, mensajes y descubrimiento en red).

Este documento presenta el diseño de **Phineas**: el problema que resuelve, el stack tecnológico, el hardware disponible y las mejoras opcionales, la asignación de pines, el mecanismo de captura, la arquitectura de software con la maniobra de entrega, la convivencia con Ferb en el espacio compartido, la comunicación con los otros robots y la simulación en Webots. Esta es la versión 3: incorpora las decisiones validadas en Webots, y el historial de cambios al final resume qué cambió respecto de la versión 2. Cada decisión se acompaña de su motivo y de la alternativa descartada, y los datos de hardware provienen de hojas de datos o del modelo de simulación del grupo.

Planteamiento del Problema

Contexto

La pista es un tablero blanco de unos 1,93 × 1,22 m con un contorno de cinta negra, y dentro de él un cuadro de unos 24 cm de lado, también de cinta negra, que funciona como zona de transferencia entre los robots móviles y el brazo. Sobre ella hay cubos de 3 × 3 × 3 cm, todos

del mismo tamaño, en seis colores. **Phineas y Ferb comparten todo el espacio**: no hay línea divisoria ni zonas asignadas, por lo que pueden encontrarse en cualquier punto. El trabajo se reparte por color: Phineas recoge los cubos rojos, naranjas y amarillos, y Ferb los verdes, azules y morados. Perry recibe cada cubo en el cuadro y lo deposita en el contenedor que corresponde. La evaluación es colectiva y debe registrarse cuántos cubos entregó cada robot (Ignacio, 2026a).

Problema Específico de Phineas

Phineas debe localizar los cubos de sus tres colores, capturarlos sin perderlos durante el trayecto, llevarlos al cuadro y soltarlos dentro, todo en un espacio donde Ferb se mueve al mismo tiempo y donde solo un robot puede usar el cuadro a la vez. Su chasis mecanum usa motores TT de reducción 1:120 sin encoders (LewanSoul, s. f.), es decir, sin medición directa de cuánto avanzó, de modo que no hay odometría (estimación de la posición integrando el giro de las ruedas). La posición de cada robot la aporta una **cámara cenital** que lee un marcador ArUco colocado encima de él, y el resto de la navegación es **reactiva**: la cámara a bordo busca los cubos y un único sensor de línea delantero detecta el borde.

Objetivos

Objetivo General

Diseñar un robot móvil autónomo sobre Raspberry Pi 5 y ROS 2 que recolecte los cubos de sus colores y los entregue en la zona de transferencia, conviviendo con Ferb en la misma pista y coordinado con Perry sin intervención humana.

Objetivos Específicos

1. Integrar el hardware disponible (Raspberry Pi 5, HAT Cokoino, cámara IMX219 y un sensor de línea) con una conexión eléctrica segura y sin compras adicionales.

2. Implementar la detección y clasificación por color de los cubos y la detección del borde de la pista.
3. Implementar una máquina de estados que encadene búsqueda, captura, transporte y entrega del cubo en el cuadro.
4. Evitar colisiones con Ferb mediante un filtro de seguridad que usa la pose cenital de ambos robots, la exclusión mutua del cuadro y una regla de prioridad.
5. Validar el comportamiento en Webots antes de probarlo en el robot real, con la misma interfaz de ROS 2.

Restricciones y Supuestos

Tabla 1

Restricciones y supuestos de diseño

Restricción o supuesto	Origen	Consecuencia en el diseño
Evitar cortocircuitos; conectar todo con los equipos apagados; operar bajo supervisión de Wilfredo	Reglas del laboratorio	Fusible e interruptor maestro en la línea de batería; montaje verificado antes de energizar
Raspberry Pi 5 y ROS 2 en cada robot; las Pi se comunican entre sí sin PC central	Enunciado del proyecto	Coordinación distribuida; el árbitro del cuadro vive en la Pi de Perry
Phineas y Ferb comparten toda la pista, sin línea divisoria	Enunciado del proyecto	Filtro de seguridad con la pose de ambos robots, prioridad fija y llave del cuadro
Reparto por colores: Phineas rojo, naranja y amarillo; Ferb verde, azul y morado	Decisión del grupo; por aprobar con el profesor	Clasificación de color por HSV con la cámara; pala gris para no confundirla con un cubo
Cubos de 3 × 3 × 3 cm, todos del mismo tamaño	Dato del grupo; modelo de Webots	Pala fija en U de 40 mm de ancho y 48 mm de fondo
Solo el material disponible: Pi 5, cámara IMX219, HAT con su batería 2S, 4 motores TT y un sensor de línea	Inventario del equipo; no se compran más piezas	Un único sensor de línea, delantero: el robot solo se traslada hacia adelante
Motores TT 1:120 de 3–6 V, sin encoders	Ficha del kit (LewanSoul, s. f.)	Pose desde la cámara cenital; ciclo de trabajo del PWM limitado a 6 V / V_{bat}
HAT documentado para Pi 4B y anteriores, no para Pi 5	Manual de Cokoino (s. f.-b)	Portar el control a lgpio y verificarlo en banco

Descripción General del Sistema

La Figura 1 muestra el flujo de un cubo. Phineas y Ferb son idénticos en hardware y software; los diferencia solo su archivo de configuración (nombre, colores asignados y prioridad). El cuadro es el único recurso que deben compartir por turnos, y la pista es un espacio común donde pueden cruzarse.

Figura 1

Flujo de un cubo a través de los tres robots



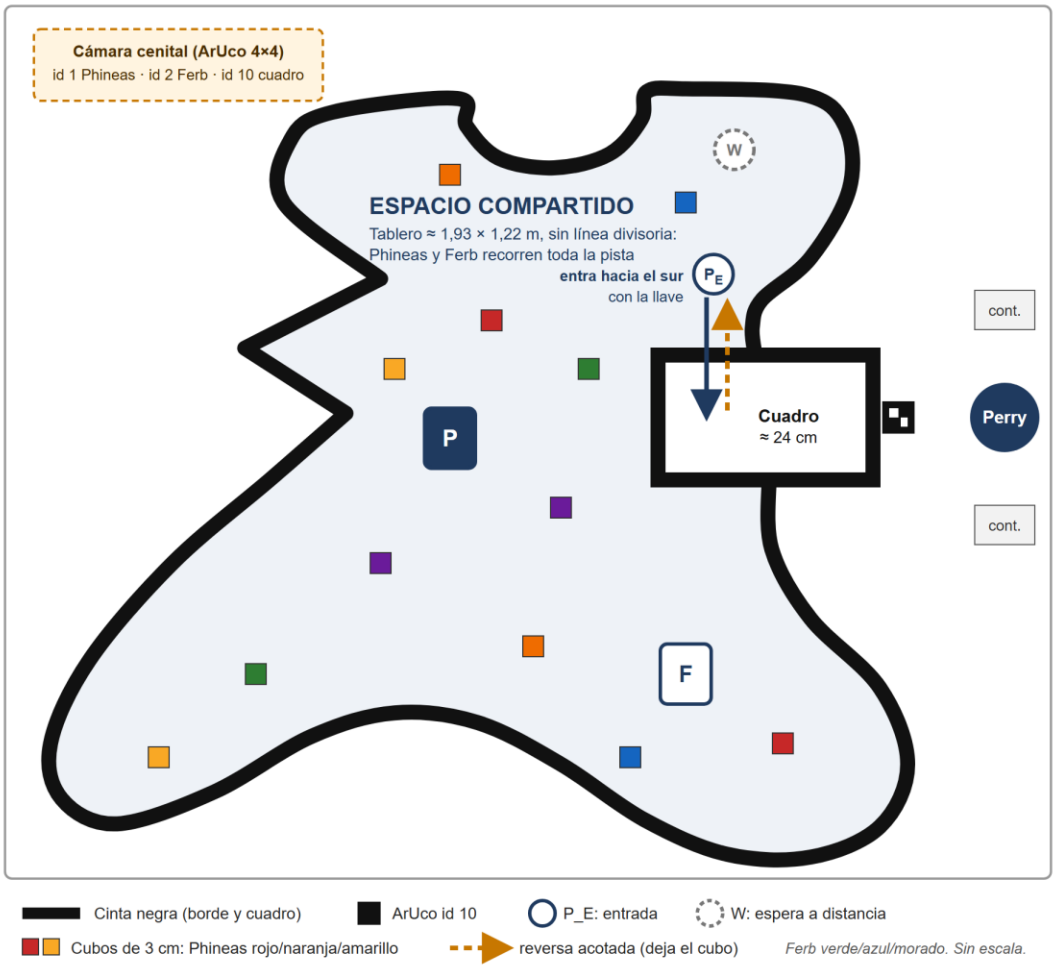
Nota. Elaboración propia.

La Figura 2 muestra la pista compartida. Los dos robots buscan en toda la superficie y nadie espera al otro para moverse: cuando se encuentran, los separa el filtro de seguridad descrito

en la sección de convivencia. El cuadro se usa por turnos, porque mide unos 24 cm y cada robot ocupa unos 19 × 16 cm sin contar la pala, así que no caben dos ni queda espacio para el brazo. Para entregar, el robot va a un punto de entrada P_E al norte del cuadro, pide la llave y entra hacia el sur; si la llave está ocupada, espera a distancia (W). La posición de los robots y del cuadro la da una cámara cenital que lee marcadores ArUco (códigos visuales cuadrados que la cámara identifica y de los que estima la posición): id 1 encima de Phineas, id 2 encima de Ferb e id 10 junto al cuadro.

Figura 2

Espacio compartido, punto de entrada y maniobra de entrega



Nota. Trazado aproximado a partir de la fotografía de la pista; sin escala. Las medidas del tablero (≈ 1,93 × 1,22 m) y del cuadro (≈ 24 cm) provienen del mundo de Webots del grupo.

Stack Tecnológico Propuesto

La Tabla 2 resume el stack por capa. El criterio común es minimizar el tiempo de depuración de un equipo que aprende ROS 2 mientras construye el robot, por eso se prefieren las opciones con instalación oficial y documentación abundante.

Tabla 2

Stack tecnológico de Phineas

Capa	Elección	Motivo	Alternativa descartada
Sistema operativo	Ubuntu Server 24.04 LTS (arm64) en la Pi; Ubuntu 24.04 en WSL2 en el PC de desarrollo	Plataforma Tier 1 de ROS 2 Jazzy: se instala por paquetes oficiales (Open Robotics, 2024)	Raspberry Pi OS: sin binarios oficiales de ROS 2; exige Docker o compilar
Middleware	ROS 2 Jazzy Jalisco (LTS hasta mayo de 2029)	Tópicos, servicios y acciones entre las tres Pi; herramientas de depuración incluidas	Protocolo propio sobre MQTT: habría que construir descubrimiento, QoS y herramientas
Transporte (RMW)	Fast DDS por defecto; `rmw_zenoh_cpp` como respaldo	Zenoh no depende de multicast (ros2, s. f.)	—
Lenguaje	Python 3.12 con `rclpy`	Decisiones a pocos hercios; OpenCV y NumPy ya ejecutan en C/C++	C++: solo para un nodo que no cumpla su latencia
GPIO	`lgpio`: PWM por software en los pines IN del HAT	Compatible con el RP1 de la Pi 5 (GPIO Zero, s. f.; joan2937, s. f.)	`RPi.GPIO`, usado por el ejemplo de Cokoino: no funciona en Pi 5
Cámara	IMX219 de 120° por CSI (cable de 22 pines incluido)	Gran angular: ve la pala y el entorno a la vez (Waveshare, s. f.-a)	Webcam USB: ocupa puerto y presupuesto de corriente USB
Visión	OpenCV: HSV, contornos y ArUco (DICT_4X4_50)	Clasificación por color sin entrenamiento; ArUco para la pose cenital	YOLO: requiere un conjunto de datos etiquetado
Localización	Cámara cenital con ArUco encima de cada robot (id 1 Phineas, id 2 Ferb; id 10 junto al cuadro)	Sin encoders no hay odometría; una cámara fija da la pose de ambos robots en el marco de la mesa	ArUco laterales vistos por la cámara del otro robot (v2): no cubre lo que queda detrás ni a los lados
Simulación	Webots R2025a en Windows + `webots_ros2` desde WSL2	Controladores externos; `Ros2Supervisor` publica `/clock`; mismos tópicos que el robot real	Gazebo: habría que modelar todo desde cero
Código y sincronía	Git + GitHub: repositorio privado `alejandrovielma/ucab-multirobot`; chrony	Un cambio de contrato llega a los tres robots en un commit; marcas de tiempo coherentes	Un repositorio por subsistema
Pruebas	`pytest` y `colcon test` en Docker con la imagen de ROS 2 Jazzy	La lógica pura y el arranque de los nodos se prueban sin robot y sin instalar ROS 2 en Windows	Probar solo en el robot: lento y arriesgado

Quedan fuera a propósito Nav2 y SLAM, porque la pista es pequeña y la pose ya la da la cámara cenital: un planificador de rutas y un mapa añadirían complejidad sin resolver un

problema actual. También quedan fuera las redes neuronales de detección, porque requieren datos etiquetados que hoy no existen. Una red YOLO puede añadirse después sin cambiar el resto del sistema si publica el mismo mensaje (Ignacio, 2026b).

Arquitectura de Hardware

Componentes Disponibles y Mejoras Futuras

El equipo usará solo el material que ya tiene, sin compras adicionales. La Tabla 3 lista ese inventario y la Tabla 4 las piezas que quedan como mejoras futuras, con el motivo por el que hoy no hacen falta. El modelo de Webots usa los mismos sensores que el robot real (cámara, un sensor de línea delantero y la pose cenital), de modo que lo que se prueba en simulación corresponde a piezas reales.

Tabla 3

Componentes de Phineas (inventario disponible)

Componente	Función	Estado
Raspberry Pi 5, 8 GB, con Active Cooler	Cómputo y nodos ROS 2; evitar reducción de frecuencia por calor	Disponible
HAT Cokoino 4WD (2 × DRV8833 + regulador GS2678)	Puentes H para 4 motores; 5,1 V para la Pi	Disponible; prueba en banco con lgpio pendiente
Batería 2S del HAT	Alimentación (7,4 V nominal, 8,4 V cargada)	Disponible
Chasis mecanum LewanSoul/Hiwonder con 4 motores TT 1:120	Movimiento	Disponible
Cámara Pi5-IMX219 de 120° por CSI	Detectar los cubos y su color	Disponible; captura en Ubuntu pendiente
Sensor de línea TCRT5000 digital (uno, delantero)	Detectar el borde de la pista y la cinta del cuadro	Disponible
Pala fija en U impresa en PLA gris	Capturar y retener un cubo	Por imprimir
Marcador ArUco superior impreso (id 1, DICT_4X4_50)	Que la cámara cenital ubique a Phineas	Por imprimir

Nota. La cámara cenital no forma parte de Phineas: será la de Perry si el profesor lo aprueba o, si no, una cámara dedicada sobre la pista. La decisión está pendiente.

Tabla 4

Mejoras futuras (opcionales, no requeridas)

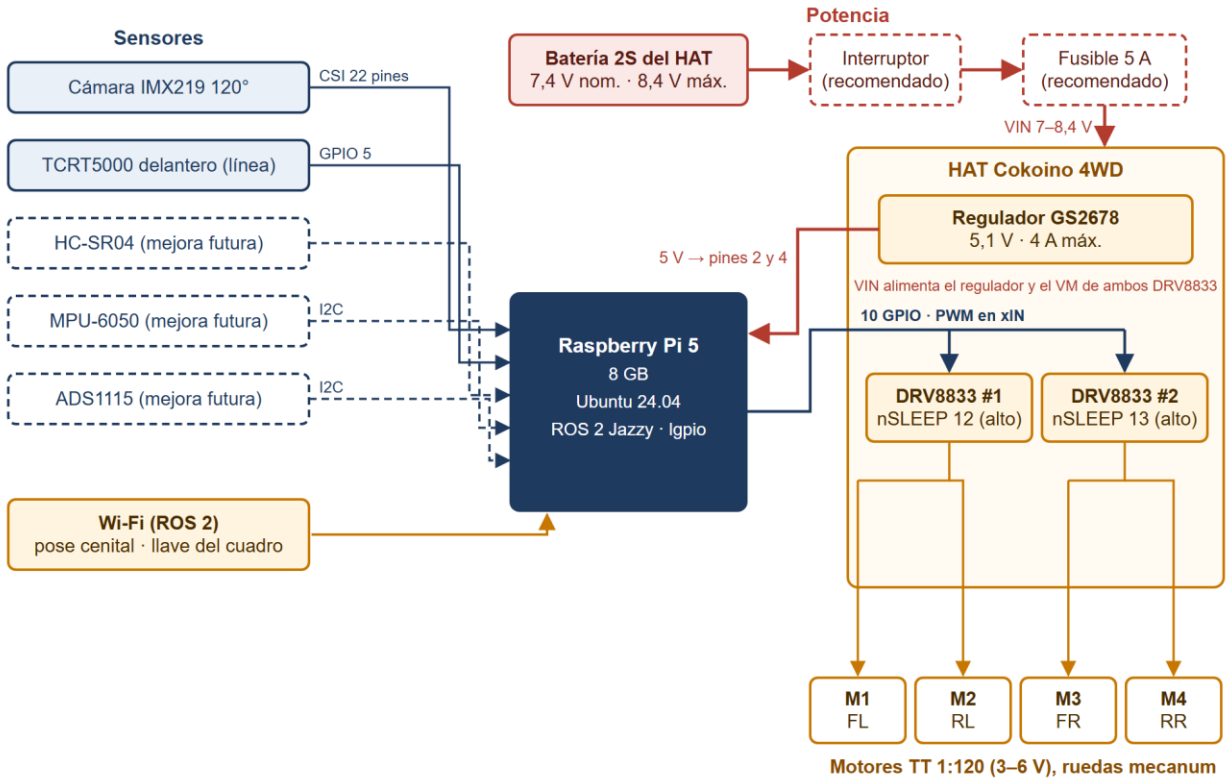
Componente	Para qué serviría	Por qué no hace falta hoy
HC-SR04 frontal (SparkFun Electronics, s. f.)	Detectar obstáculos sin depender de la pose cenital	El filtro de seguridad usa la pose de ambos robots
IMU MPU-6050	Mantener el rumbo entre lecturas de la cámara	La pose cenital ya incluye el rumbo
Marcadores ArUco laterales	Que la cámara a bordo identifique al otro robot	La evasión usa la pose cenital
Servo SG90 en GPIO18 con PWM hardware del RP1 (Gadgetoid, s. f.)	Levantar la pala para soltar el cubo	La pala fija suelta el cubo retrocediendo
Sensor IR FC-51 (Cirkit Designer, s. f.)	Confirmar que hay un cubo en la pala	La captura se decide con la posición del cubo en la imagen
Sensor de color TCS34725 (Waveshare, s. f.-b)	Confirmar el color de cerca	La cámara clasifica el color; la pala gris evita confusiones
Convertidor ADC ADS1115	Medir el voltaje de la batería (la Pi no tiene ADC)	Recomendado, no bloqueante
Interruptor maestro y fusible de 5 A	Corte de energía y protección ante cortocircuitos	Recomendado por seguridad; se instala si el laboratorio lo provee

Diagrama de Bloques

La Figura 3 separa la ruta de potencia (rojo), que lleva la batería a los puentes H y al regulador, de la ruta de señal (azul). El regulador GS2678 del HAT entrega 5,1 V y hasta 4 A a la Pi por los pines 2 y 4 del cabezal, y los dos DRV8833 toman VM directamente de la batería (Cokoino, s. f.-b). La cámara usa el conector CSI propio de la Pi y no ocupa pines del cabezal. La pose cenital y la llave del cuadro llegan por Wi-Fi como tópicos y servicios de ROS 2.

Figura 3

Diagrama de bloques de hardware de Phineas



Nota. Los bloques con borde discontinuo son mejoras futuras o elementos recomendados. Elaboración propia a partir del esquemático de Cokoino (s. f.-a).

Restricciones Eléctricas Verificadas

- **Solo batería 2S.** El DRV8833 opera entre 2,7 V y 10,8 V, con máximo absoluto de 11,8 V (Texas Instruments, 2015). Una batería 3S (12,6 V cargada) destruye los drivers aunque el HAT acepte 12 V.
- **Ciclo de trabajo limitado.** Los motores TT están especificados para 3–6 V y reciben el voltaje de la batería. El PWM no debe superar `duty_max = 6 V / V_bat` (71 % con la batería llena); `hat_motor_driver` aplica ese tope con `motor_max_voltage: 6.0` y `battery_voltage: 8.4`.

- **Presupuesto de 5 V.** El regulador del HAT da 4 A, por debajo de los 5 A recomendados para la Pi 5; sin negociación USB-PD la Pi limita los periféricos USB a 600 mA (Raspberry Pi Ltd, s. f.). La cámara CSI no consume de ese presupuesto USB; no debe activarse `usb_max_current_enable`.
- **Corriente por puente.** El DRV8833 del HAT es el encapsulado TSSOP, que admite 500 mA RMS y 2 A de pico por puente, con protección por sobrecorriente entre 2 A y 3,3 A (Texas Instruments, 2015). Un motor bloqueado contra Ferb o el borde puede dispararla; el filtro de seguridad reduce ese caso deteniendo el avance ante la línea y ante el otro robot.
- **Rampas de aceleración.** El arranque de cuatro motores hunde la tensión de la batería y puede reiniciar la Pi y corromper la microSD. Conviene limitar la variación de velocidad por ciclo y medir el pin 2 con multímetro durante un arranque brusco.

Mecanismo de Recolección

El chasis por sí solo no retiene un cubo: en cualquier giro el cubo se escapa. La Tabla 5 compara las opciones con el criterio que más pesa en este proyecto: retener el cubo sin actuadores que el equipo no tiene.

Tabla 5

Comparación de mecanismos de captura

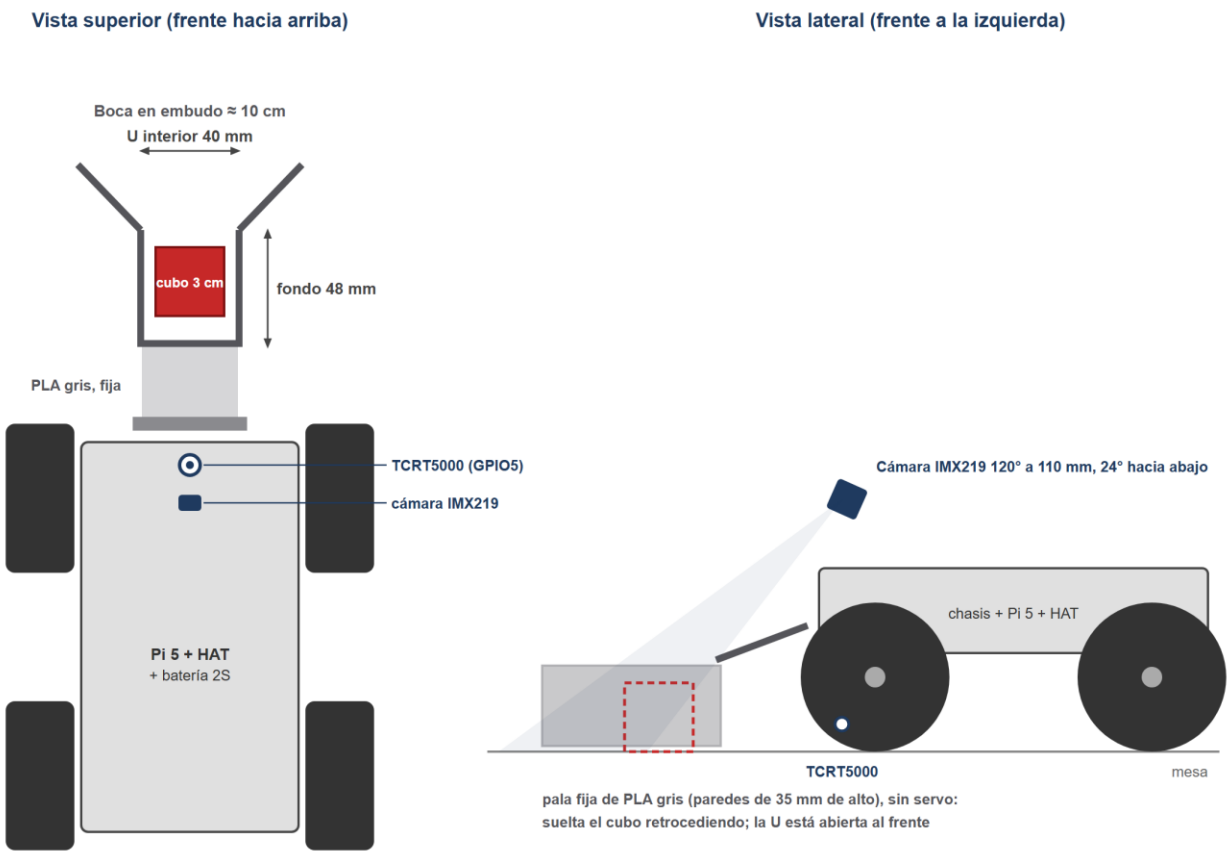
Criterio	Empuje directo	Pala fija en U	Pala en U con servo (v2)	Pinza servoaccionada
Retención	Nula	Alta al avanzar y en arco; un giro en el sitio la expulsa	Alta	Total
Alineación requerida	Alta	Baja: el embudo de unos 10 cm guía el cubo	Baja	Alta
Actuadores	0	0	1 servo	1–2 servos

Criterio	Empuje directo	Pala fija en U	Pala en U con servo (v2)	Pinza servoaccionada
Entrega en el cuadro	Retroceder	Retroceder: la U está abierta al frente	Subir la pala y retroceder	Abrir y levantar
Hardware adicional	—	Solo impresión en PLA	SG90, no disponible	Servos, no disponibles
Estado en el proyecto	—	Modelada y validada en Webots	Descartada	—

Se adopta la **pala fija en U** (Figura 4), impresa en PLA gris. Es una captura pasiva: una boca en embudo de unos 10 cm encauza el cubo hacia una U de 40 mm de ancho y 48 mm de fondo, que deja unos 5 mm de holgura a cada lado de un cubo de 30 mm. Un error de percepción de algunos centímetros no provoca un fallo de captura, porque la geometría corrige la alineación. Para soltar el cubo, el robot retrocede: la U está abierta al frente, así que el cubo queda donde está. Sin servo no hay piezas móviles ni pines ocupados; a cambio, el robot necesita retroceder para entregar, lo que obliga a una excepción acotada en el filtro de seguridad (sección de software). Esta decisión sería la equivocada si los cubos variaran de tamaño o si hubiera que levantarlos; eso último lo hace Perry.

Figura 4

Pala fija en U y ubicación de la cámara y del sensor de línea



Nota. Medidas del modelo RecolectorUCAB de Webots (U de 40 × 48 mm, paredes de 35 mm de alto, cubo de 30 mm).

La pala impone dos condiciones al software, ambas observadas en Webots:

- **Color gris.** Con la pala naranja, la cámara de Phineas detectaba sus propias alas como un cubo naranja y las perseguía. El gris tiene saturación baja, y el detector lo descarta igual que al fondo blanco y a la línea negra. Una prueba automática impide volver a pintar la pala de un color de cubo.
- **Giros en arco con cubo.** Un giro en el sitio expulsó el cubo a unos 45°; en arco, con el robot avanzando, el cubo se mantiene (comprobado con control por teclado). Por eso, con

cubo, `safety_gate` impone un avance mínimo de 0,05 m/s y un giro máximo de 0,6 rad/s.

Asignación de Pines y Conexiones

El HAT ocupa diez pines fijos: GPIO12 y GPIO13 para nSLEEP, y GPIO16, 17, 22, 23, 24, 25, 26 y 27 para las entradas de los puentes H (Cokoino, s. f.-b). El único sensor adicional es el de línea, en GPIO5. La Tabla 6 y la Figura 5 muestran la asignación, que coincide con `config/phineas.yaml`. En la Pi 5 el cabezal pertenece al chip del RP1 y se maneja con `lgpio` (`gpio_chip: 4` en el YAML, a confirmar con `gpioinfo`).

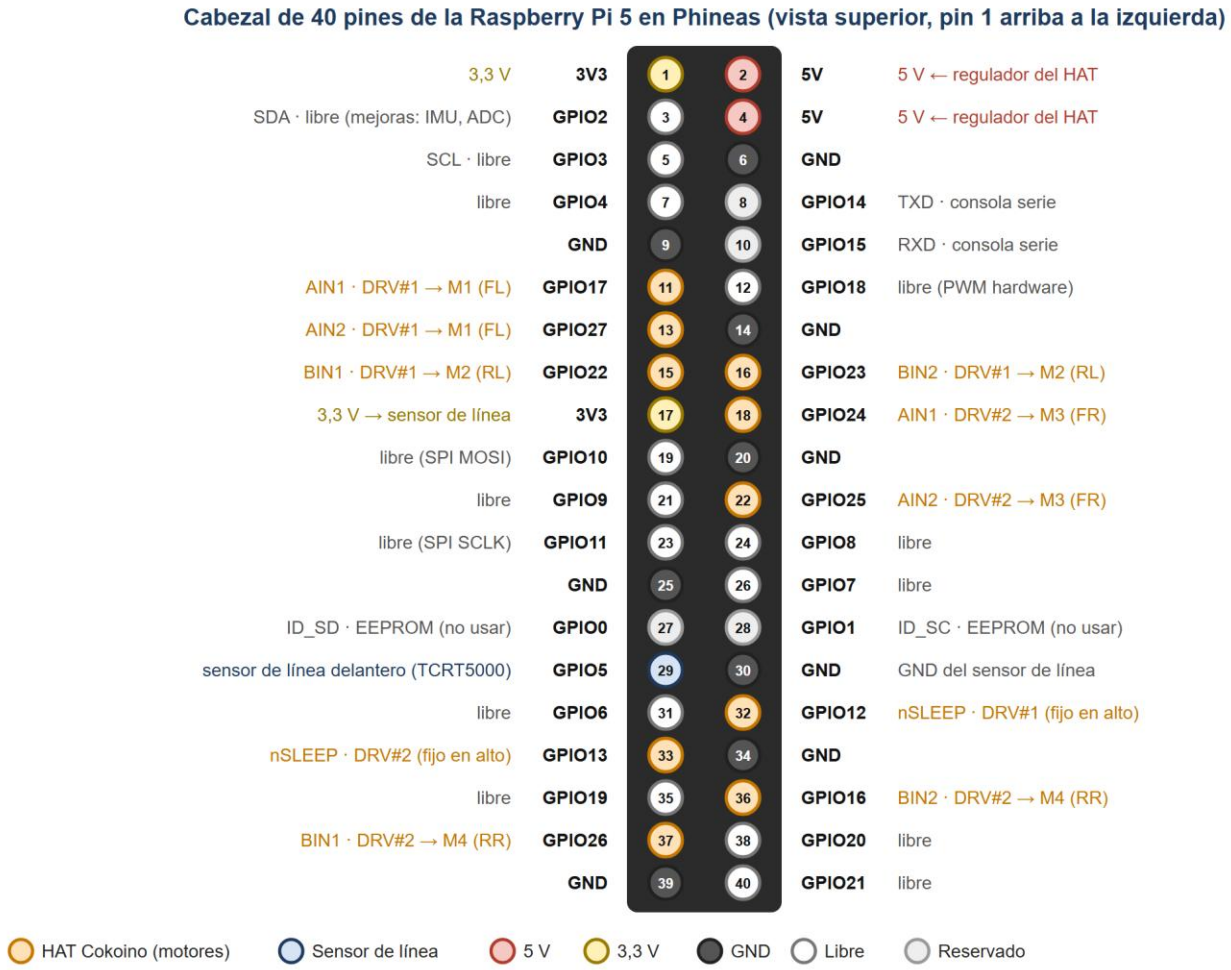
Tabla 6

Asignación de pines de la Raspberry Pi 5 en Phineas

Función	GPIO (BCM)	Pin físico	Nota
nSLEEP de DRV8833 #1 / #2	12 / 13	32 / 33	Fijos en alto
Motor delantero izquierdo (FL): IN1 / IN2	17 / 27	11 / 13	DRV8833 #1, canal A (M1)
Motor trasero izquierdo (RL): IN1 / IN2	22 / 23	15 / 16	DRV8833 #1, canal B (M2)
Motor delantero derecho (FR): IN1 / IN2	24 / 25	18 / 22	DRV8833 #2, canal A (M3)
Motor trasero derecho (RR): IN1 / IN2	26 / 16	37 / 36	DRV8833 #2, canal B (M4)
Sensor de línea delantero (TCRT5000, salida digital)	5	29	Módulo a 3,3 V; alto = línea negra (`active_level: 1`)
Cámara IMX219	—	Conector CSI	No usa el cabezal
I2C (SDA / SCL)	2 / 3	3 / 5	Libre; reservado para mejoras (IMU, ADC)
Consola serie (reservado)	14 / 15	8 / 10	Depuración sin red

Figura 5

Mapa del cabezal de 40 pines de Phineas



Nota. Pines del HAT según el manual de Cokoino (s. f.-b); asignación tomada de config/phineas.yaml.

Control de los Motores

El código de ejemplo del fabricante regula la velocidad aplicando PWM de 1 kHz sobre nSLEEP (Cokoino, s. f.-a). La hoja de datos indica que el puente tarda hasta 1 ms en activarse después de que nSLEEP sube (t_{WAKE}), que es exactamente un periodo a 1 kHz, así que la relación entre ciclo de trabajo y velocidad resulta errática. Además, un solo nSLEEP por chip obliga a que dos ruedas giren siempre a la misma velocidad.

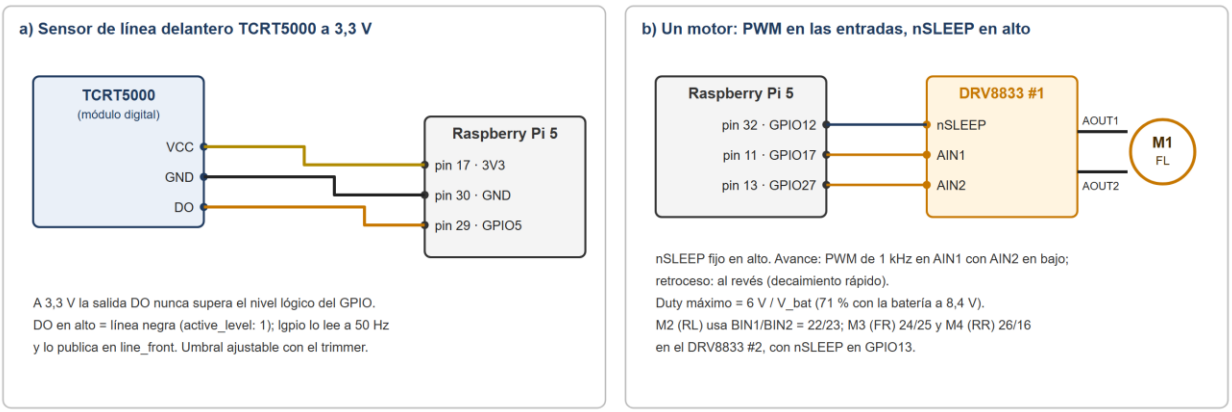
La corrección es mantener nSLEEP fijo en alto y aplicar el PWM sobre las entradas xIN1/xIN2, algo que la hoja de datos admite de forma explícita: PWM en xIN1 con xIN2 en bajo produce avance con decaimiento rápido, y a la inversa para retroceder (Texas Instruments, 2015). Las entradas aceptan 2 V como nivel alto, por lo que bastan los 3,3 V de la Pi. Así cada rueda tiene velocidad independiente, que es lo que exige la cinemática mecanum. El nodo `hat_motor_driver` implementa este esquema con PWM de 1 kHz de `lgpio`, el tope de ciclo de trabajo de 6 V / V_{bat} y un *watchdog* de 300 ms. Si el PWM por software de lgpio no alcanzara, el plan de respaldo es PWM de baja frecuencia sobre nSLEEP; como cada lado del robot está en un chip distinto, se conserva el giro diferencial.

Conexión del Sensor de Línea y de los Motores

La Figura 6 detalla las dos conexiones que hace el equipo. El sensor de línea se alimenta a 3,3 V para que su salida digital nunca supere el nivel lógico de los pines de la Pi, que trabajan a 3,3 V (Raspberry Pi Ltd, s. f.). Cada motor se cablea como el del panel b, con los pines de la Tabla 6.

Figura 6

Conexión del sensor de línea y de un motor



Nota. Todas las conexiones se realizan con la batería desconectada, conforme a las reglas del laboratorio.

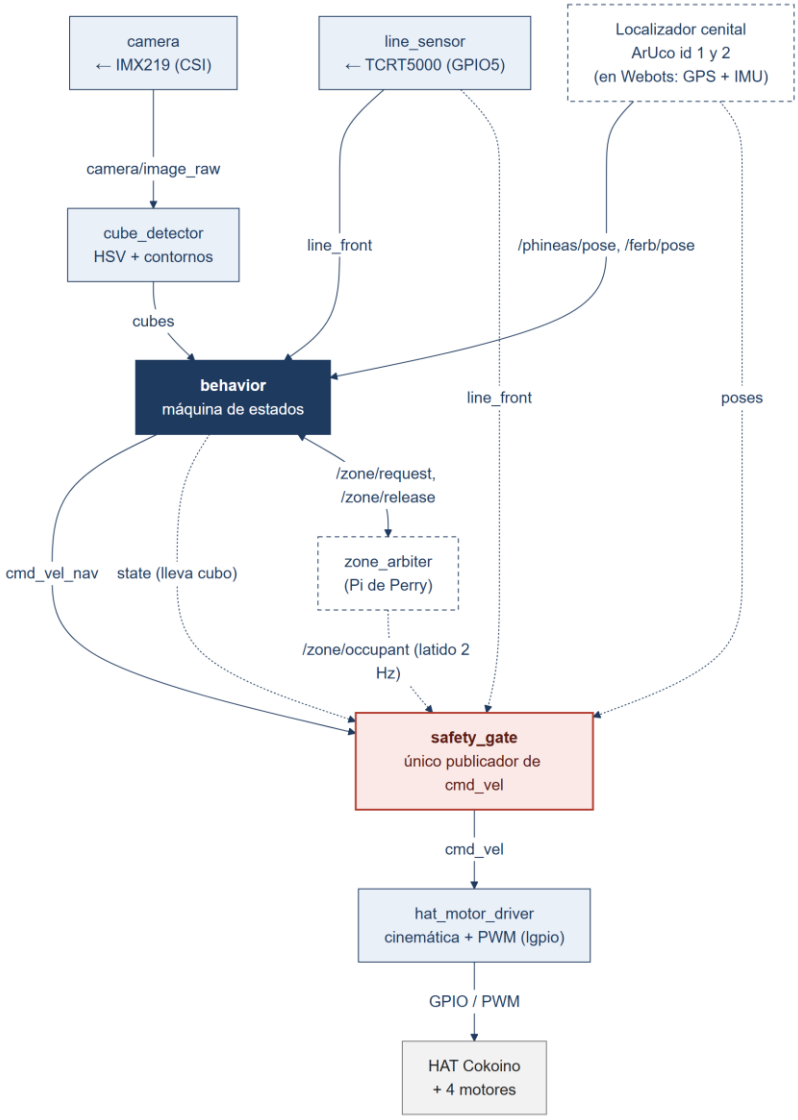
Arquitectura de Software

Nodos de ROS 2

El software de Phineas vive en el repositorio privado de GitHub `alejandrovielma/ucab-multirobot`, en el paquete `mobile_base`, que comparte con Ferb; solo cambian el archivo de parámetros (`config/phineas.yaml` o `config/ferb.yaml`) y el espacio de nombres (`/phineas` o `/ferb`). Hay un nodo por responsabilidad (Figura 7): los sensores publican hechos (hay línea negra al frente, hay un cubo rojo en tal posición de la imagen), el localizador cenital publica la pose de cada robot, la máquina de estados decide, y un único nodo publica la orden que llega a los motores. Los nodos que deciden son los mismos en Webots y en el robot real; solo cambia la capa de hardware.

Figura 7

Grafo de nodos y tópicos de Phineas



Nota. Los tópicos locales llevan el prefijo /phineas/, que se omite en el diagrama. Elaboración propia a partir de docs/architecture.md del repositorio.

El nodo `safety_gate` aplica el patrón **safety filter** (filtro de seguridad): `behavior` publica un comando deseado en `cmd_vel_nav`, y `safety_gate` es el único publicador de `cmd_vel`. Así ninguna lógica nueva, por ejemplo la entrega, puede saltarse las reglas. En cada ciclo de 20 Hz aplica, en este orden: parada total si no llega el latido de `/zone/occupant` en 2 s o si la pose propia tiene más de 0,5 s; eliminación de toda traslación lateral y de todo avance

negativo; anulación del avance si hay línea negra al frente; frenado y giro ante el otro robot; y límites de giro con cubo. A su vez, `hat_motor_driver` detiene los motores si no recibe órdenes durante 300 ms (**dead-man switch**, o parada por inactividad).

Percepción de Cubos por Color

El nodo `cube_detector` convierte la imagen de la cámara a HSV, separa los píxeles de saturación alta del fondo blanco y busca contornos. Conserva solo los que tienen área de cubo, proporción cercana a 1 y solidez alta; ese filtro descarta la línea negra, que es alargada, el contorno del cuadro, que es hueco, y la pala gris, que tiene poca saturación. A cada contorno le asigna un color según su tono, y solo los rojos, naranjas y amarillos pasan a ser objetivos de Phineas. La máquina de estados usa la posición del cubo en la imagen: lo centra girando y lo da por alcanzado cuando su centro baja de la fila 0,85 de la altura (`capture_cy`).

Una lente gran angular como la de 120° deforma la imagen hacia los bordes, por lo que la cámara se calibra una vez con un tablero de ajedrez. Los rangos de color viven en el archivo YAML y se recalibran con la luz real del laboratorio, que el simulador no reproduce.

Cinemática Mecanum

El nodo `hat_motor_driver` (y el plugin `RecolectorDriver` en Webots) convierte la velocidad deseada del robot (v_x hacia adelante, v_y hacia la izquierda, ω_z de giro) en la velocidad de cada rueda con el modelo cinemático inverso de Taheri et al. (2015), donde r es el radio de la rueda (32,5 mm) y l_x, l_y son las semidistancias entre ejes (60,5 mm y 65,5 mm):

$$\omega_{FI} = (v_x - v_y - (l_x + l_y)\,\omega_z) / r \qquad \omega_{FD} = (v_x + v_y + (l_x + l_y)\,\omega_z) / r$$
$$\omega_{TI} = (v_x + v_y - (l_x + l_y)\,\omega_z) / r \qquad \omega_{TD} = (v_x - v_y + (l_x + l_y)\,\omega_z) / r$$

Sin encoders, estas velocidades se aplican en lazo abierto, normalizadas para que ninguna rueda supere `duty_max`, y el rumbo se corrige con la pose cenital. Aunque la cinemática admite v_y, el filtro de seguridad lo anula: con un único sensor de línea delantero, desplazarse de lado es

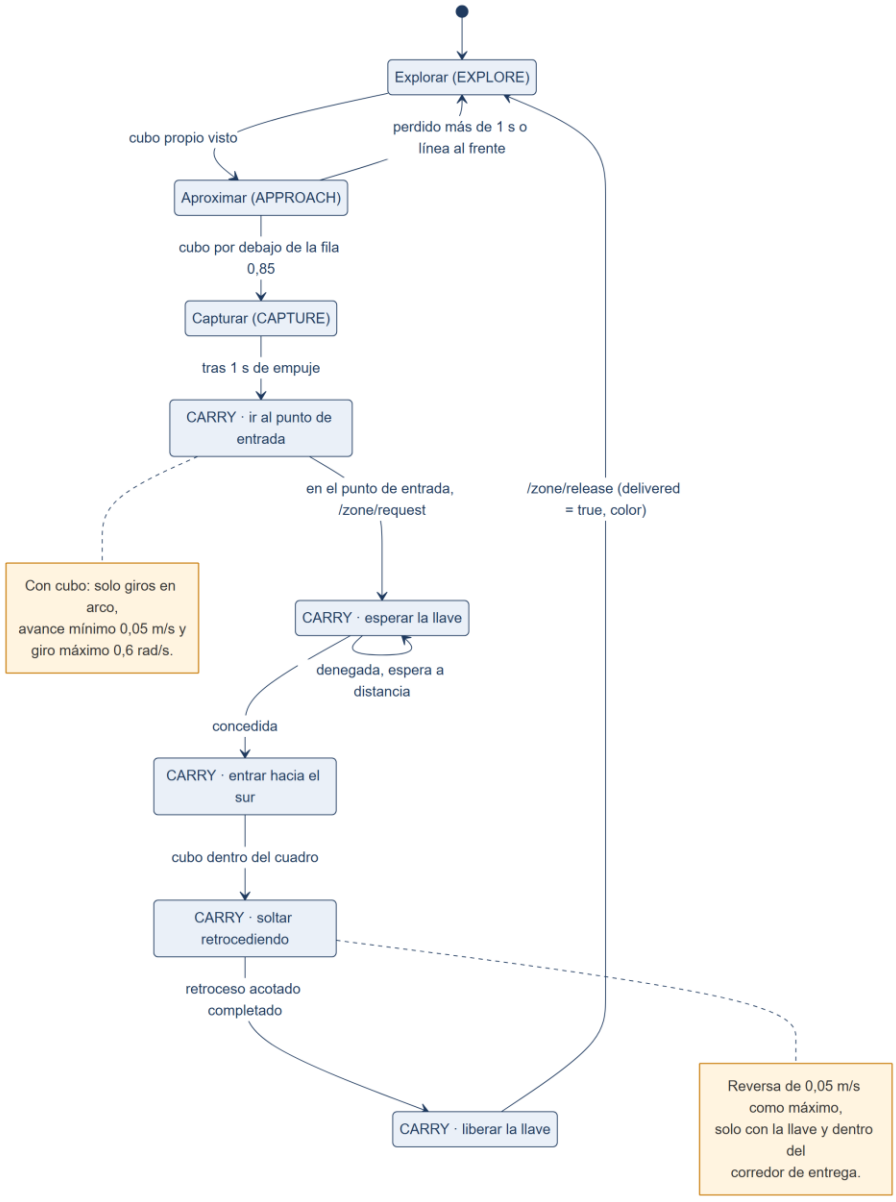
ciego al borde. Los signos de las ruedas se validan con una prueba de desplazamiento lateral sin el filtro (`sim.launch.py logic:=false`), en Webots y luego en el robot.

Máquina de Estados

El nodo `behavior` implementa la **finite state machine** (máquina de estados finita) de la Figura 8. En **EXPLORE** avanza a 0,12 m/s y, al ver la línea negra, gira en el sitio durante un tiempo aleatorio; en **APPROACH** se orienta hacia el cubo propio más grande de la imagen; en **CAPTURE** empuja 1 s para meterlo en la pala, y en **CARRY** lo transporta y lo entrega. Cada estado tiene un criterio de salida medible y un tiempo máximo, de modo que el robot nunca queda esperando indefinidamente un evento que no llega.

Figura 8

Máquina de estados de Phineas



Nota. Los umbrales son valores de config/phineas.yaml. Los estados de CARRY se implementan en la rama feat/carry-delivery. Elaboración propia.

Maniobra de Entrega

La entrega es la parte del diseño que más cambió frente a la versión 2, porque la pala fija solo suelta el cubo retrocediendo y el filtro de seguridad prohíbe retroceder. Se resuelve con una **reversa acotada bajo la llave** del cuadro, en estos pasos:

1. El robot, con el cubo en la pala, va al punto de entrada P_E, al norte del cuadro, girando solo en arco.
2. Pide la llave con `/zone/request`. Si se la niegan, espera a distancia y vuelve a pedirla.
3. Con la llave, entra rumbo al sur cruzando la cinta norte del cuadro hasta que el cubo queda dentro.
4. Retrocede a 0,05 m/s como máximo y una distancia acotada. La U abierta al frente deja el cubo en el cuadro.
5. Libera la llave con `/zone/release`, indicando `delivered = true` y el color; el árbitro devuelve el conteo oficial, y el robot vuelve a EXPLORE.

La excepción vive en `safety_gate` y **no confía en la máquina de estados**: permite cruzar la cinta del cuadro y retroceder solo si su propia pose está dentro del corredor de entrega y `/zone/occupant` indica que este robot tiene la llave. Si la máquina de estados fallara, el filtro seguiría impidiendo la reversa fuera del cuadro. Se descartaron dos alternativas: girar en el sitio para expulsar el cubo, porque en Webots sale a unos 45° en una dirección que no se controla y la diagonal del robot (unos 0,25 m) supera el interior del cuadro (unos 0,22 m); y salir en arco hacia adelante, porque la U arrastra el cubo consigo.

Convivencia en el Espacio Compartido

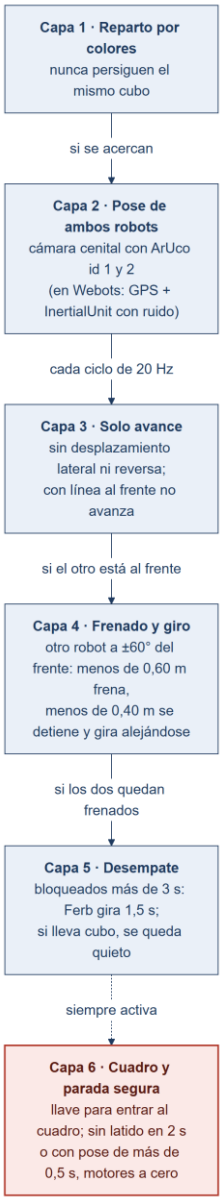
La consigna es que Phineas y Ferb anden libremente por toda la pista al mismo tiempo. Para evitarse, cada robot necesita saber dónde está el otro. La versión 2 proponía que cada robot viera al otro con su cámara, gracias a marcadores ArUco laterales, y lo esquivara desplazándose de lado. La versión 3 lo reemplaza por **posiciones compartidas**: una cámara cenital lee el ArUco superior de cada robot (id 1 Phineas, id 2 Ferb, diccionario DICT_4X4_50) y publica la pose de ambos en `/phineas/pose` y `/ferb/pose`. Será la cámara de Perry si el profesor lo aprueba o, si

no, una cámara dedicada sobre la pista; esa decisión está pendiente. En Webots, la cámara se emula con un GPS y una InertialUnit en cada robot, con ruido gaussiano opcional.

El diseño apila seis capas (Figura 9). Cada una reduce los casos que llegan a la siguiente, y un choque solo ocurre si fallan todas; el enfoque se conoce como **defense in depth** (defensa en profundidad).

Figura 9

Capas de convivencia entre Phineas y Ferb



Nota. Valores de config/phineas.yaml, validados en Webots. Elaboración propia a partir del esquema de capas del documento inicial del grupo (Ignacio, 2026a).

Solo Avanzar

El robot tiene un único sensor de línea, en el frente. Desplazarse de lado o retroceder es, por lo tanto, ciego al borde de la pista: en Webots, el esquive lateral de la versión 2 sacó a los dos robots del tablero. Por eso `safety_gate` elimina toda traslación lateral y todo avance

negativo, y anula el avance cuando hay línea negra al frente; girar en el sitio sigue permitido cuando el robot no lleva cubo. La única excepción es la reversa acotada de la entrega.

Frenado y Giro ante el Otro Robot

Con la pose de ambos, `safety_gate` calcula la distancia entre centros y el ángulo al otro robot. Si el otro está a menos de 60° a cada lado del frente, reduce el avance linealmente entre 0,60 m y 0,40 m, se detiene por debajo de 0,40 m y gira alejándose a 0,6 rad/s. El umbral de 0,40 m sale de la geometría: las puntas de las palas se tocan a unos 0,38 m entre centros. Si los dos quedan bloqueados de frente más de 3 s, el robot **sin prioridad**, Ferb, gira 1,5 s para salir; si lleva un cubo, se queda quieto, porque girar en el sitio lo expulsaría de la pala. Sin una regla asimétrica ambos cederían a la vez y volverían a bloquearse, un **livelock** (bloqueo activo).

Las dos últimas capas no dependen de que el otro robot coopere: si se pierde el latido del árbitro durante más de 2 s, o si la pose propia tiene más de 0,5 s de antigüedad, los motores van a cero. Cada robot publica además su estado en `/phineas/state` o `/ferb/state`, que `safety_gate` usa para saber si lleva un cubo.

Tabla 7

Alternativas de evasión consideradas

Alternativa	Resultado	Motivo
Esquive lateral con las mecanum (v2)	Descartada	Ciego al borde con un solo sensor delantero; en Webots sacó a los dos robots de la pista
Percepción local con ArUco laterales y la cámara a bordo (v2)	Mejora futura	La cámara no ve detrás ni a los lados; exige cuatro marcadores por robot
Ultrasonido HC-SR04 frontal y trasero (v2)	Mejora futura	No forma parte del inventario; la pose cenital cubre el caso
Pose cenital + frenado, giro y prioridad (v3)	Adoptada	Validada en Webots; un solo sensor en cada robot basta

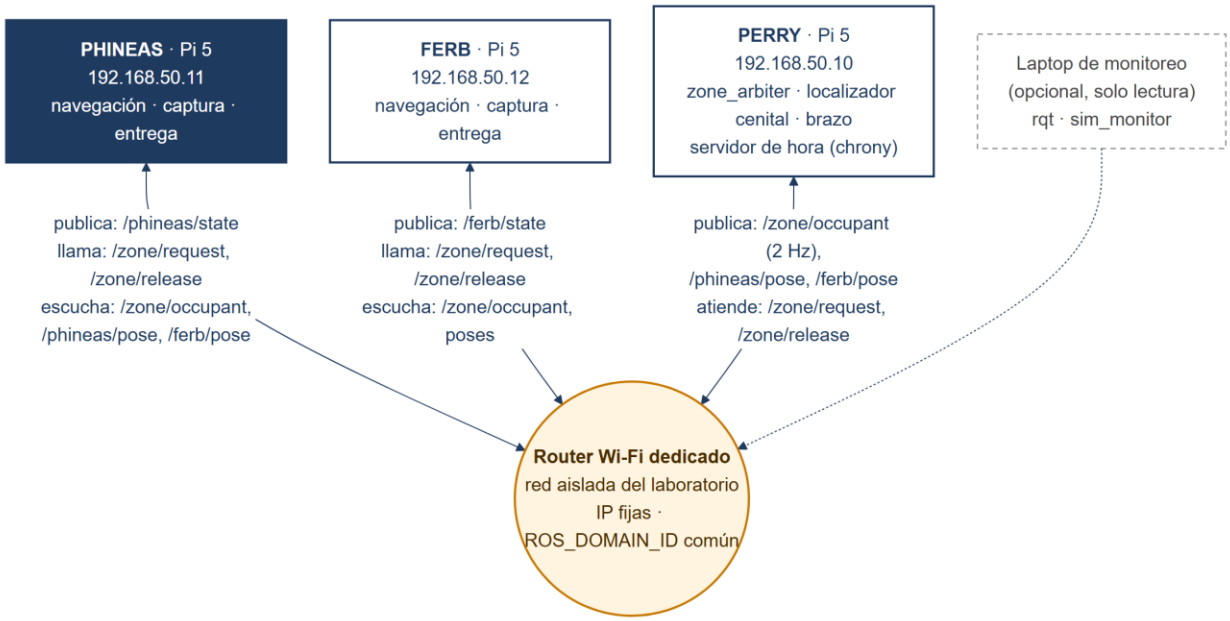
Comunicación con Ferb y Perry

Red y Descubrimiento

Las tres Pi se conectan a un router Wi-Fi dedicado con direcciones IP fijas y el mismo `ROS_DOMAIN_ID` (Figura 10). El descubrimiento de DDS usa multicast, que muchas redes universitarias bloquean, y el síntoma es que los robots se ven en casa y no en el laboratorio sin ningún error visible. Si el multicast falla también en el router propio, las tres Pi cambian a `rmw_zenoh_cpp`, que descubre a los pares mediante un router Zenoh (ros2, s. f.); el cambio debe hacerse en las tres a la vez, porque dos RMW distintos no se comunican. Perry actúa además como servidor de hora con chrony; los relojes sincronizados son obligatorios, porque el filtro de seguridad descarta las poses con más de 0,5 s.

Figura 10

Topología de red y tópicos publicados por cada robot



Nota. Las direcciones IP son un ejemplo de plan de direccionamiento. Elaboración propia.

Contrato de Interfaces de Phineas

La Tabla 8 fija los mensajes que Phineas intercambia con el resto del sistema. La calidad de servicio, o QoS (Quality of Service), se elige según el uso: el ocupante del cuadro usa **transient local**, que entrega el último valor a quien se conecta tarde, para que un robot reiniciado sepa de inmediato quién tiene la llave.

Tabla 8

Interfaces ROS 2 de Phineas con el resto del sistema

Interfaz	Tipo	Rol de Phineas	QoS / uso
/phineas/pose, /ferb/pose	multirobot_interfaces/RobotPose	Suscriptor	Las publica el localizador cenital; caducan a 0,5 s
/phineas/state	multirobot_interfaces/RobotState	Publica	Estado de la máquina; indica si lleva cubo
/zone/request	multirobot_interfaces/RequestZone (servicio)	Cliente	granted, holder y duración de la llave (60 s)
/zone/release	multirobot_interfaces/ReleaseZone (servicio)	Cliente	Envía delivered y el color; devuelve delivered_total
/zone/occupant	std_msgs/String	Suscriptor	Transient local; latido de 2 Hz; vacío = libre
/phineas/cmd_vel	geometry_msgs/Twist	Lo publica solo safety_gate	Lo consume hat_motor_driver o el plugin de Webots

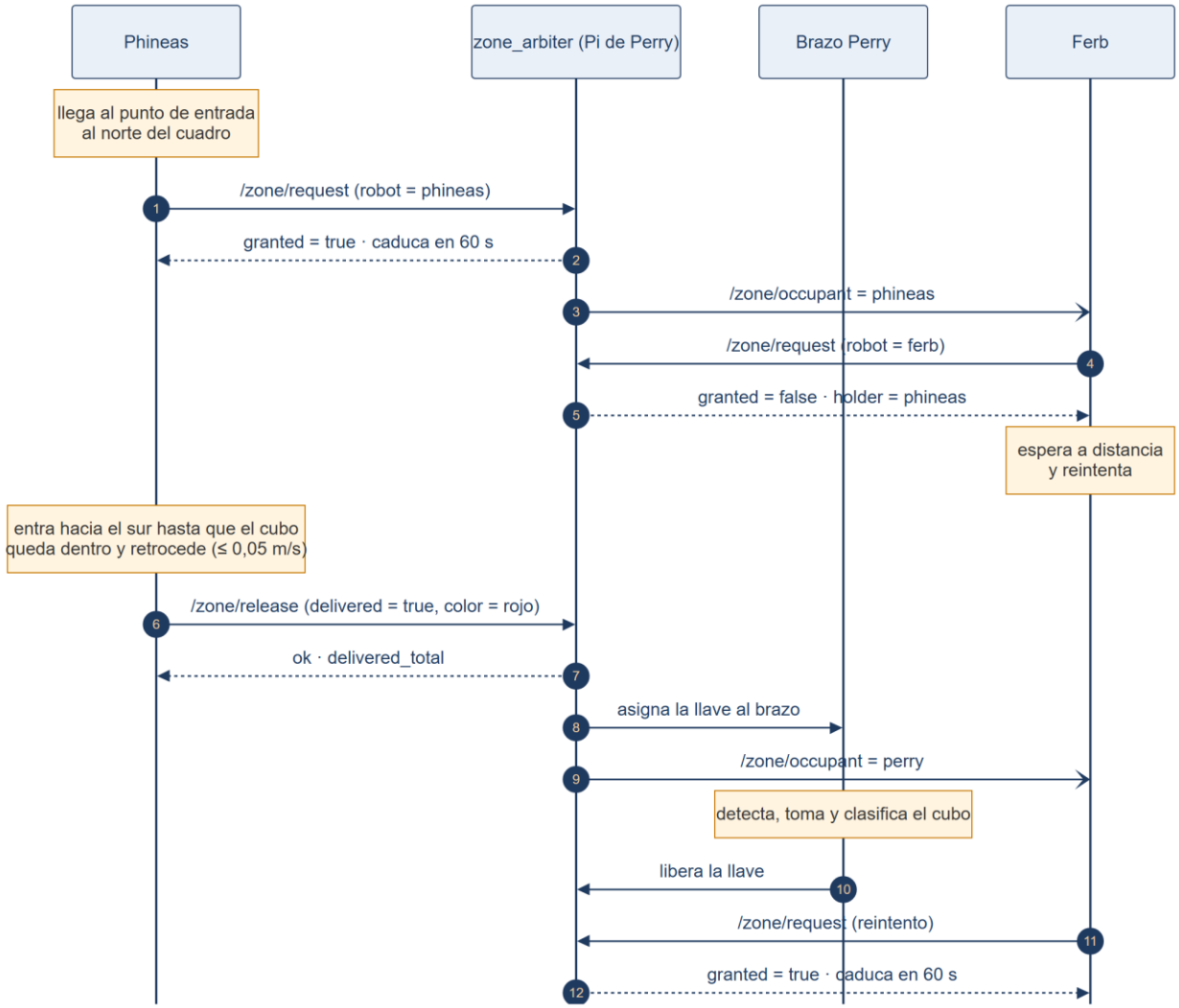
Nota. Contrato vigente del repositorio (docs/architecture.md); reemplaza los nombres en español del contrato inicial del grupo (Ignacio, 2026a).

Protocolo de Acceso a la Zona de Transferencia

El cuadro es un recurso de exclusión mutua: solo un robot o el brazo puede usarlo a la vez. Se gestiona con un **lease** (préstamo con caducidad) que concede `zone_arbiter` en la Pi de Perry, la única que está fija y enchufada. La Figura 11 muestra un caso con contención.

Figura 11

Secuencia de acceso al cuadro con contención entre Phineas y Ferb



Nota. Adaptado del mecanismo de llave del documento inicial del grupo (Ignacio, 2026a), con los nombres del repositorio.

- La llave **caduca a los 60 s**: un robot averiado con la llave no paraliza el sistema.
- Phineas **libera la llave después de retroceder**, con el cubo ya dentro del cuadro, no al llegar; si la liberara antes, Ferb podría entrar mientras Phineas aún maniobra.
- El árbitro publica `/zone/occupant` a 2 Hz y ese mensaje funciona como **heartbeat** (latido): sin él durante 2 s, `safety_gate` detiene el robot.

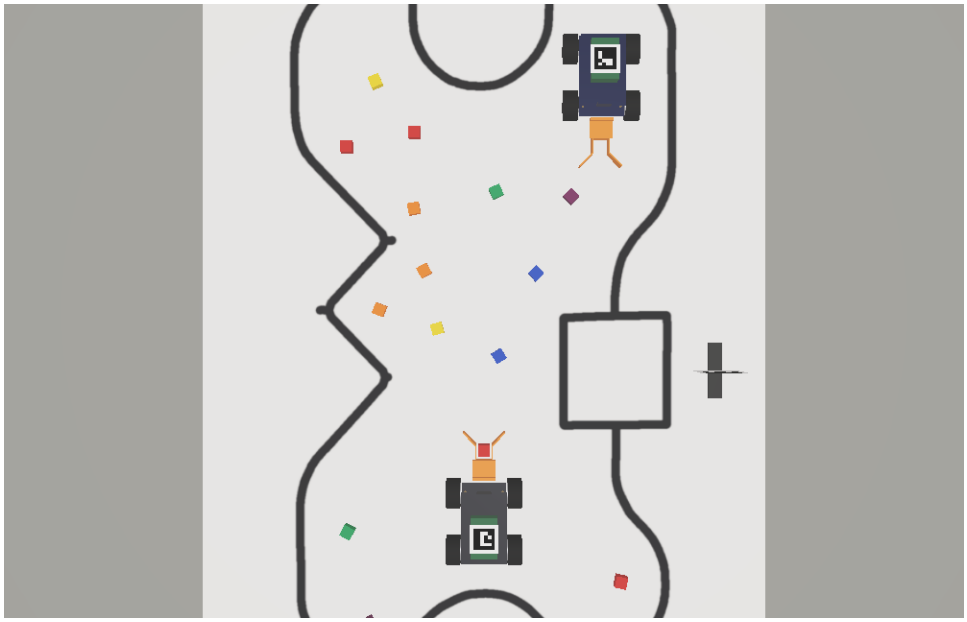
- El **conteo oficial de cubos** lo lleva el árbitro al recibir `/zone/release` con `delivered = true`, y lo devuelve en la respuesta. Una sola fuente de verdad evita contadores desalineados si un robot se reinicia.

Simulación en Webots

El grupo modeló la pista (tablero de 1,93 × 1,22 m con contorno de cinta negra y cuadro de unos 24 cm), catorce cubos de 3 cm y los dos recolectores en Webots R2025a (Figura 12). El modelo reproduce el chasis de 186 × 161 mm con ruedas de 65 mm, los motores sin encoder, la pala fija en U gris y los sensores que usa el robot real (Equipo Phineas, 2026). Las ruedas mecanum se simulan con fricción direccional a 45°, una técnica que también usa el modelo del KUKA youBot incluido en Webots, y patinan algo al girar, como las reales.

Figura 12

Vista cenital del mundo de Webots con Phineas y Ferb



Nota. Captura del mundo pista_ucab.wbt del grupo, anterior al cambio de color de la pala: en el modelo actual la pala es gris. Tomado de Equipo Phineas (2026).

La Tabla 9 relaciona cada dispositivo simulado que usan los nodos de ROS 2 con su pieza real.

Tabla 9

Dispositivos del modelo de Webots y su equivalente real

Dispositivo en Webots	Pieza real	Qué devuelve
motor_fl, motor_fr, motor_rl, motor_rr	Motores TT por el HAT Cokoino	Velocidad en rad/s (máximo 10,5)
servo_pala	Ninguna: la pala es fija	Se mantiene en 0
camara	IMX219 de 120° (el modelo usa todavía 54°)	Imagen de 640 × 480 en bgra8, publicada en camera/image_raw
linea_frente	TCRT5000 delantero en GPIO5	Negro si el valor baja de line_threshold; se publica en line_front
gps + imu_sim (InertialUnit)	Cámara cenital con ArUco	Pose en el marco de la mesa, con ruido gaussiano opcional; se publica en pose

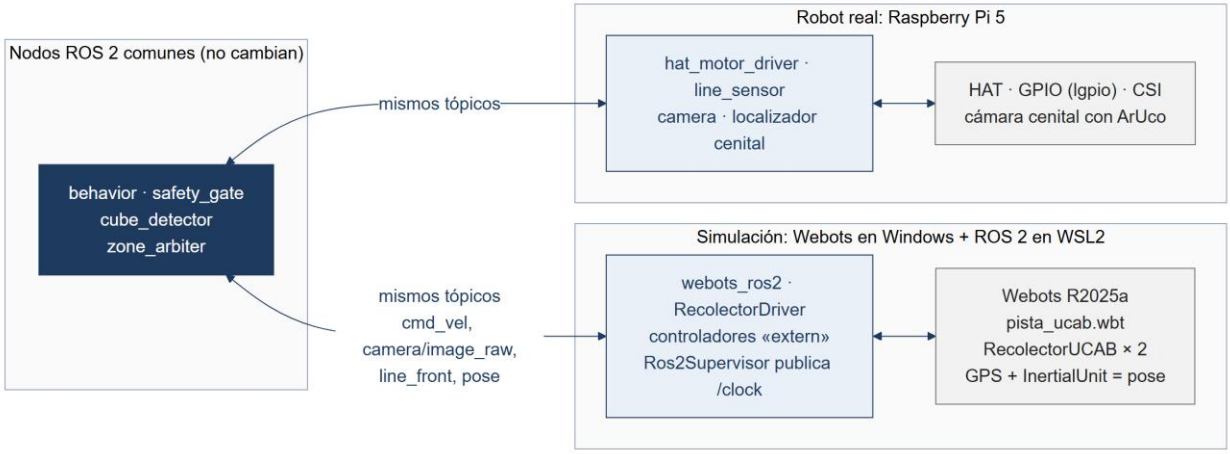
Nota. El proto conserva dispositivos del plan inicial (cuatro sensores de línea, ultrasonido, sensores en la pala, acelerómetro y giroscopio) que los nodos de ROS 2 no usan (Equipo Phineas, 2026).

Del Simulador al Robot Real

El puente con ROS 2 es el paquete `webots_ros2`. En Windows, Webots R2025a se ejecuta en el sistema anfitrión y ROS 2 Jazzy dentro de WSL2 con Ubuntu 24.04, y el paquete se instala con `ros-jazzy-webots-ros2` (Open Robotics, s. f.). Los dos robots usan controladores externos (`<extern>`) conectados por el plugin `RecolectorDriver`, y `Ros2Supervisor` publica el reloj `/clock` de la simulación; sin él, los nodos con tiempo de simulación no avanzan. La Figura 13 muestra la consecuencia de diseño: los nodos de decisión y percepción son los mismos en ambos entornos, y solo se reemplaza la capa que habla con el hardware.

Figura 13

Mismos nodos de ROS 2 sobre Webots y sobre el robot real



Nota. Elaboración propia.

La simulación sirve para validar la lógica: la máquina de estados, el filtro de seguridad, el protocolo de la llave y los encuentros con Ferb. No reproduce bien el patinaje real de las ruedas sobre el tablero, los colores con la luz del laboratorio, la caída de voltaje de la batería ni el calentamiento de los motores. Por eso cada prueba se aprueba primero en Webots y después se repite en la pista real. La lógica pura (`mobile_base/core`) no importa ROS 2, OpenCV ni lgpio, y se prueba con `pytest` en un contenedor Docker con ROS 2 Jazzy, sin robot. Existe además un problema conocido de conexión entre WSL2 y Webots en Windows (cyberbotics, s. f.), que se resuelve desactivando el DNS tunneling de WSL.

Webots ya corrigió cuatro decisiones de la versión 2: la pala fija expulsa el cubo si el robot gira en el sitio, la pala naranja se detectaba como un cubo, el esquive lateral sacaba a los robots de la pista y la distancia de parada original no contaba el largo de la pala.

Riesgos y Verificaciones Pendientes

Tabla 10

Riesgos principales de Phineas

Riesgo	Mitigación	Verificación
El HAT no responde en la Pi 5	Portar el control a lgpio; si falla, pedir otro driver	Prueba de banco: cada motor gira en el sentido esperado
La cámara CSI no funciona bien en Ubuntu 24.04	Plan B: Raspberry Pi OS con ROS 2 en Docker	Capturar una imagen desde Ubuntu el primer día
Cámara cenital no aprobada o mal calibrada	Cámara dedicada; homografía con al menos 4 puntos; sin pose fresca el robot se detiene	Error menor a 2 cm y 5° en 5 posiciones medidas
Choques con Ferb en el espacio compartido	Filtro con la pose de ambos; parada a 0,40 m; prioridad fija	Cruces de frente y en ángulo en Webots y en la pista
Salida de la pista con un solo sensor de línea	Solo avance; giros en el sitio sin cubo	5 min de movimiento sin cruzar el borde
El cubo se sale de la pala	Con cubo, solo giros en arco ($\geq 0{,}05$ m/s, $\leq 0{,}6$ rad/s)	Trayectorias de CARRY en Webots y en la pista
La reversa de entrega sale del cuadro	Reversa $\leq 0{,}05$ m/s y acotada, solo con la llave y dentro del corredor; el filtro lo comprueba con su propia pose	Entrega completa observada en Webots
Confusión de colores por la luz del laboratorio	Calibración HSV en YAML	20 imágenes por color; rojo y morado no se confunden
La pala detectada como cubo	Pala gris, con prueba automática que impide pintarla de un color de cubo	Imagen con la pala y Ferb en el encuadre
Reinicios de la Pi por caída de tensión	Rampas de aceleración; plan B: convertidor de 5 V y 5 A	Medir el pin 2 en un arranque brusco
Cubos reales distintos de 3 cm	La U de 40 mm deja unos 5 mm por lado; medir antes de imprimir	Medición con calibrador
Diferencias entre Webots y el robot real	Validar cada prueba en ambos entornos	Comparar tiempos y tasas de éxito

Plan de Pruebas

Cada requisito tiene una prueba observable que se ejecuta primero en Webots y después en la pista real. Los criterios provienen del documento de requisitos del grupo (Ignacio, 2026a), ajustados al reparto por colores, al espacio compartido y a la pala fija.

Tabla 11

Pruebas de aceptación de Phineas

ID	Requisito	Criterio de aceptación
RF-01	No sale de la pista	5 min de movimiento continuo sin cruzar el borde
RF-02	Localiza cubos de sus colores	Encuentra 3 de 5 cubos propios en 3 min con cubos ajenos presentes
RF-03	Ignora los cubos ajenos	Ningún cubo verde, azul o morado llega al cuadro en 10 intentos
RF-04	Entrega el cubo en el cuadro	8 de 10 cubos quedan dentro del cuadro y el conteo oficial aumenta
RF-05	Pide turno antes de entrar	Con la llave denegada, espera a distancia y no entra
RF-06	Se retira tras entregar	Retrocede a 0,05 m/s como máximo dentro del corredor y libera la llave con delivered = true
RF-07	Evita a Ferb	10 cruces con Ferb sin contacto
RF-08	Se detiene sin enlace	Al apagar el router, se detiene en menos de 2 s
RF-09	Se detiene sin pose	Al tapar su ArUco o detener el localizador, se detiene en menos de 1 s
RF-10	Retiene el cubo	10 transportes hasta el cuadro sin perder el cubo

Conclusiones

Phineas puede construirse con el material que el equipo ya tiene: Raspberry Pi 5, HAT Cokoino con su batería 2S, cuatro motores TT, cámara IMX219 de 120° y un sensor de línea, más una pala fija impresa en PLA gris. Las dos correcciones más importantes frente al material del fabricante siguen vigentes: usar solo baterías 2S, por el límite de 10,8 V del DRV8833, y mover el PWM de nSLEEP a las entradas de los puentes.

Tener un solo sensor de línea, delantero, fija el resto del diseño: el robot solo avanza o gira, la evasión se basa en la pose cenital de ambos robots en vez de en esquives laterales, y la entrega necesita una reversa corta que el filtro de seguridad solo concede con la llave y dentro del corredor del cuadro. Webots ya corrigió cuatro decisiones antes de arriesgar el hardware. Quedan pendientes la decisión sobre la cámara cenital, la implementación y validación de la entrega, y la prueba de banco del HAT y de la cámara sobre la Pi 5 con Ubuntu.

Historial de Cambios

Tabla 12

Historial de cambios del documento

Versión	Fecha	Cambios
2	4 de octubre de 2026	Espacio compartido sin zonas; evasión por percepción local con ArUco laterales y esquive lateral; seis capas de convivencia
3	12 de octubre de 2026	Cubos de 3 × 3 × 3 cm. Pala fija en U de 40 × 48 mm, gris y sin servo; SG90, FC-51 y TCS34725 pasan a mejoras futuras. Inventario real sin compras: un solo sensor de línea (GPIO5) y pines de config/phineas.yaml. Pose por cámara cenital con ArUco (decisión de la cámara pendiente). Solo avance, frenado y giro ante el otro robot en lugar del esquive lateral. Giros en arco con cubo. Entrega con reversa acotada bajo la llave. Stack: WSL2, webots_ros2, repositorio y pruebas en Docker. Pista de 1,93 × 1,22 m sin línea divisoria

Referencias

Cirkit Designer. (s. f.). *How to use sensor FC-51 IR: Pinouts, specs, and examples*. https://docs.cirkitdesigner.com/component/794e0aa1-9872-489b-bdc1-5f95d749086a/sensor-fc-51-ir

Cokoino. (s. f.-a). *CKK0018: Pi Power & 4WD HAT* [Repositorio de código, esquemático y ejemplos]. GitHub. https://github.com/Cokoino/CKK0018

Cokoino. (s. f.-b). *Introduction of Cokoino 4WD Robot Hat* [Manual técnico]. GitHub. https://github.com/Cokoino/CKK0018/tree/main/Tutorial

cyberbotics. (s. f.). *Webots ROS2 program running in Ubuntu (Windows WSL2) is not able to connect to Webots host simulator in Windows 11* (Issue #1031) [Reporte de error]. GitHub. https://github.com/cyberbotics/webots_ros2/issues/1031

Equipo Phineas. (2026). *Simulación de los recolectores en Webots* [Modelo de simulación y documentación inédita]. Robótica, Universidad Católica Andrés Bello.

Gadgetoid. (s. f.). *Hardware PWM on the Raspberry Pi 5* [Gist]. GitHub. https://gist.github.com/Gadgetoid/b92ad3db06ff8c264eef2abf0e09d569

GPIO Zero. (s. f.). *API - Pins*. https://gpiozero.readthedocs.io/en/latest/api_pins.html

Ignacio. (2026a). *Sistema multirrobot colaborativo: Documento inicial* [Documento inédito]. Robótica, Universidad Católica Andrés Bello.

Ignacio. (2026b). *Sistema multirrobot: Tecnología, código y to-do de laboratorio* [Documento inédito]. Robótica, Universidad Católica Andrés Bello.

joan2937. (s. f.). *lgpio: Python module documentation*. https://abyz.me.uk/lg/py_lgpio.html

LewanSoul. (s. f.). *LewanSoul mecanum robot chassis car kit with 360° omnidirectional wheel, aluminum alloy frame, TT motor* [Página de producto]. Amazon. https://www.amazon.com/dp/B093WDD9N5

Open Robotics. (s. f.). *Installation (Windows)* [Webots]. ROS 2 Documentation: Jazzy. https://docs.ros.org/en/jazzy/Tutorials/Advanced/Simulators/Webots/Installation-Windows.html

Open Robotics. (2024, 23 de mayo). *ROS 2 Jazzy Jalisco released*. https://www.openrobotics.org/blog/2024/5/ros-jazzy-jalisco-released

Raspberry Pi Ltd. (s. f.). *Raspberry Pi hardware*. Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

ros2. (s. f.). *rmw_zenoh: RMW for ROS 2 using Zenoh as the middleware* [Repositorio de código]. GitHub. https://github.com/ros2/rmw_zenoh

SparkFun Electronics. (s. f.). *HC-SR04 ultrasonic sensor* [Hoja de datos]. Digi-Key. https://www.digikey.com/htmldatasheets/production/1979760/0/0/1/hc-sr04.html

Taheri, H., Qiao, B., & Ghaeminezhad, N. (2015). Kinematic model of a four mecanum wheeled mobile robot. *International Journal of Computer Applications, 113*(3), 6–9. https://doi.org/10.5120/19804-1586

Texas Instruments. (2015). *DRV8833 dual H-bridge motor driver* (SLVSAR1E) [Hoja de datos]. https://www.ti.com/lit/ds/symlink/drv8833.pdf

Waveshare. (s. f.-a). *IMX219 camera module for Raspberry Pi 5, 8MP, MIPI-CSI interface, options for 79.3° / 120° FOV*. https://www.waveshare.com/pi5-imx219.htm

Waveshare. (s. f.-b). *TCS34725 color sensor* [Wiki]. https://www.waveshare.com/wiki/TCS34725_Color_Sensor